Twin Neighbor Embedding: A Pre-Embedding Transform for Improved Network Visualization.

Alexandros Athanasiadis

Nikos Pitsianis
Faculty Supervisor

Department of Electrical and Computer Engineering
Aristotle University of Thessaloniki

Abstract. This thesis aims to discuss challenges in existing methods for the visual analysis of networks and describe the Twin Neighbor Embedding (TNE) method, a simple pre-embedding transform designed to improve the quality of existing network layout algorithms, based on unpublished related research [1]. Network representations play a central role in scientific data analysis, as many real-world systems and relationships are naturally modeled as networks. Network visualization is therefore an integral component of network analysis, as spatial layouts can reveal structural properties of the underlying network in an intuitive and interpretable manner. However, existing network layout methods exhibit key limitations that can obscure structural relationships and hinder accurate interpretation of the data. SNE based methods have been applied in recent years to the layout of sparse networks using efficient and scalable algorithms [2]. However methods like SG-t-SNE still exhibit key limitations. We identify three recurring issues which the Twin Neighbor Embedding method aims to address: (a) vertices with small graph-theoretic distance or strong edge weights are often placed excessively close in the layout, leading to visual crowding and partial collapse of local substructure; (b) certain vertices or groups of vertices are erroneously placed close together, forming hallucinatory clusters that suggest relationships not present in the data; and (c) conventional layout methods do not explicitly encode geometric edge placement, which often results in excessive crossings of long edges. The Twin Neighbor Embedding method improves on these limitations by encoding vertex adjacency, edge adjacency, and vertex–edge incidence relationships in a unified augmented matrix used as a pre-layout transform. This representation is then supplied to standard layout algorithms in place of the traditional adjacency matrix. Despite its conceptual simplicity, the proposed transform improves layout quality across a variety of real-world and synthetic networks, as demonstrated through visual analysis and quantitative evaluation of neighborhood preservation, classification accuracy, vertex crowding and edge crossings.

1. Introduction

1.1. Motivation

Networks have been used and studied for centuries, as they can model a variety of systems and relationships. At their most basic, networks are a set of data points (called vertices or nodes) with interconnections between them (called edges or links). The versatility of networks enables us to gain insight into systems, prove conjectures, or develop algorithms in many scientific fields.

Network analysis and scientific network visualization are an increasingly vital part of data analytics for exploratory study and confirmatory analysis of relationships, patterns, community structures, and their evolution or dynamics in real-world data networks across various domains such as biology [3], [4], [5], [6], neurology [7], sociology [8], and related domains [9], [10].

1.2. Problem Description

A simple undirected graph G is the tuple (V, E) consisting of a set of vertices (nodes) V, and a set of edges (links) $E \subseteq V \times V$. Denote by $|V|$ the number of vertices and by $|E|$ the number of edges. If two vertices $v, u \in V$ form an edge $(v, u) \in E$ then we call v and u neighboring or adjacent, and we write $v \sim u$. A vertex v is incident to an edge e if it is part of e. Throughout this work we shall use the terms network and graph interchangeably.

Network visualization conventionally is the task of assigning each vertex $v \in V$ a position in some surface Σ. Then the edges may be drawn as lines or curves (not strictly in Σ) connecting their incident vertices. As a post-processing step, additional network properties and features may be overlaid on top of the layout as functions defined on the vertices or edges.

Consider the graph $G = (V, E)$. Each vertex $v \in V$ is mapped to a unique point $\mathbf{x}(v) \in \Sigma$. The set $X = \{\mathbf{x}(v) \mid v \in V\}$ is called the vertex layout or embedding[1] and is determined by a network layout algorithm.

The surface Σ is called the embedding space and is commonly $\mathbb{R}^2$, but sometimes $\mathbb{R}^3$ or even restricted to $\mathbb{S}^1$ or $\mathbb{S}^2$ [11], [12], [13]. For the purposes of visualization Σ is low-dimensional.

Existing software packages for scientific network visualization differ primarily in three aspects: (i) the choice of embedding space Σ, (ii) the choice of embedding criteria and layout algorithm, and (iii) post-processing utilities and overlays. The analysis in this thesis is primarily concerned with (ii), and from this point forward it will be assumed that $\Sigma \equiv \mathbb{R}^d$ (normally $d = 2$) and overlays will be used only to highlight structural aspects of the network useful in the visual understanding and qualitative assessment of the layout.

Constructing such embeddings is inherently challenging. A low-dimensional layout must represent a complex network topology while preserving structural relationships that are important for visual interpretation. Because many structural constraints cannot be simultaneously satisfied in a two-dimensional embedding, layout algorithms inevitably introduce distortions whose nature depends on the chosen embedding objective.

1.3. Related Work

Various layout algorithms exist with differing criteria for determining the spatial configuration X. Some of these methods and criteria will be discussed in more detail in Section 2.1 and Section 2.2.

A common criterion is to preserve the pairwise geodesic distances of vertices on G proportionally in the layout X. Yet it is impractical to rigidly impose all geodesic distance pairs be preserved in equal measures, considering the confined low-dimensional embedding space does not allow for the variety, irregularity, and uncertainty in sparse data networks. We prioritize that adjacent vertices in network G are placed in close proximity in the layout X. This is the Stochastic Neighbor Embedding (SNE) [14] principle applied to network topology.

However SNE-based layouts suffer from undesirable artifacts that obscure structural relationships and hinder accurate interpretation of the data. We identify three recurring issues:

[1] In topological graph theory an embedding strictly is a drawing of the vertices and edges in a way such that the edges may intersect only at their ends in the target embedding space. In this thesis embedding and layout are used interchangeably, in all cases meaning a layout.

(a) vertex crowding: vertices with small graph-theoretic distance or strong edge weights are often placed excessively close in the layout, leading to visual crowding and partial collapse of local substructure.
(b) erroneous clustering: certain vertices or groups of vertices are erroneously placed close together, forming hallucinatory clusters that suggest relationships not present in the data.
(c) excessive long-edge crossings: conventional layout methods do not explicitly encode geometric edge placement, which often results in excessive crossings of long edges.

t-SNE [15] was largely successful in alleviating the crowding issue compared to classical SNE for dimensionality reduction of point-cloud data. SG-t-SNE [2], which adapts t-SNE for sparse networks, has achieved good performance, while still exhibiting some of the limitations of SNE methods.

In an effort to address the remaining issues of SG-t-SNE for sparse networks we developed the Twin Neighbor Embedding (TNE) method and the TWIN software package as part of yet unpublished research with Dimitris Floros, Nikos Pitsianis, and Xiaobai Sun [1].

AA1: Section break here?

The aim of this thesis is to present and analyze the TNE method and the TWIN software and compare it to existing network layout methods in terms of visual quality and quantitative evaluation criteria. Additional contributions are presented in Section 1.4.

TNE is not a layout algorithm in the traditional sense, but instead a pre-layout transform of G which constructs a supra-network consisting of G, its line graph G_ℓ, and additional connections linking vertices of G to corresponding incident edges represented in G_ℓ. This is represented algebraically via a unified augmented matrix which encodes vertex and edge adjacency as well as vertex–edge incidence. This representation is then used in place of the traditional adjacency matrix to extend and improve pre-existing layout methods, primarily —but not limited to— SG-t-SNE. The method is described in detail in Section 4.

Despite its conceptual and mathematical simplicity, TNE proves effective in mitigating vertex crowding and erroneous clustering across various real-world and synthetic networks. Additionally, due to its explicit geometric encoding of edge placement it significantly alleviates the problem of excessive edge crossing. Various visualization results, as well as quantitative evaluations of the method are presented in Section 5.

Our software package TWIN allows the user to interact with the Twin Neighbor Embedding method and compare to traditional vertex embedding in an interactive GUI with various tuning parameters. The package is open source and the source code can be found at https://github.com/alex-unofficial/twin-neighbor-embedding. The TWIN software package is discussed in Section 6.

Finally, we discuss the limitations of our method as well as potential future improvements in Section 7.

1.4. Contributions

AA2: Curved edge drawing based on edge layout

AA3: Multi-level approach(?) — see Section 2.1.1: Spring-Electrical model

AA4: We'll see...

2. Background

Network diagrams have been used for centuries to analyze complex relationships, appearing as early as the 14th century [16], long before the formal development of graph theory.

Early work in graph theory was closely related to properties of drawings of graphs, such as planarity — the ability to represent a graph in the Euclidean plane without intersecting edges. Investigations of polyhedral graphs by Leonhard Euler, particularly his formula relating vertices, edges, and faces of convex polyhedra, are often cited among the earliest results connected to graph theory.

Prior to the development of computer graphics, graph drawings in the form of node–link diagrams were created by manually placing each vertex and connecting them with lines or arrows (depending on the type of graph). In most cases the effective "layout algorithm" was simply the intuition of the artist drawing the diagram, placing related vertices closer together, or arranging them in a visually meaningful manner. However, more sophisticated layout methods were occasionally used such as circular layouts appearing in the combinatorial diagrams of Ramon Llull or the top-down arrangement of medieval family-tree drawings.

AA5: Too much historical context?

While the history of graph drawing spans several centuries, the modern field of network visualization largely emerged alongside the development of computer graphics in the 20th century.

The need to represent larger, more complex networks created the demand for automatic layout generation methods. Such methods vary widely in both methodology and objectives, depending on the type of input graph, the specific use case, and the scientific intent of the user.

We restrict our scope to scientific network visualization of general undirected sparse networks. Our aim is the analysis of networks for exploratory study and visual confirmation of community structures and relationships in data. Consequently we restrict our attention to unconstrained euclidean layouts of general networks. Specialized layouts designed for specific application domains and particular graph classes —such as grid layouts for VLSI and PCB design, tree layouts for hierarchical data networks, or spatially constrained layouts like circular layouts and arc diagrams— are outside the scope of this work.

2.1. Conventional Methods

Consider a graph $G = (V, E)$. We must assign to each vertex $v \in V$ a position $\mathbf{x}(v) \in \mathbb{R}^d$.

Denote by $n = |V|$ the number of vertices and by $m = |E|$ the number of edges. Given some ordering of the vertices $1, 2, ..., n$ the layout X is made of n position vectors $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n$ such that $\mathbf{x}_i = \mathbf{x}(i)$ is the position of vertex i. From a computational perspective the layout $X \in \mathbb{R}^{n \times d}$ can be treated as a conventional matrix whose rows are the vectors $\mathbf{x}_i^T$.

It is obvious that any single graph has infinitely many layouts, given we have not stated any constraints on $\mathbf{x}(v)$. However, not every layout is of the same quality. Take, for example, n randomly sampled positions in $\mathbb{R}^d$ and assign each one to a vertex. This is a valid layout, but it encodes no information about the underlying network. From a data analysis perspective, it is practically useless.

Therefore we must define what makes a quality layout. This is not a simple question, and existing methods adopt different perspectives.

Although layout algorithms are presented in many different forms, a large portion of the literature can be interpreted through the lens of optimization. Many methods explicitly define an objective or energy function that measures the quality of a layout and then apply numerical procedures to approximately minimize it. Other methods are not explicitly presented as optimization procedures, and may describe the algorithm in isolation — as analogues of physical systems, heuristic approaches, or functions of algebraic structures associated with the graph — but can still be understood as implicitly optimizing a related criterion. It is useful in these cases, when possible, to identify the implicit objective or criteria of a method in order to properly understand its behavior and results.

In the following sections we briefly review several widely used classes of layout algorithms, focusing on representative methods and the principles that underlie their formulations.

AA6: [17] is a good resource

2.1.1. Force-directed methods

Force-directed (or force-based) methods are among the most widely used approaches for scientific network visualization. The vast majority of algorithms implemented in academic and commercial network visualization software are based on this principle. Consequently, force-based methods exhibit substantial variation in theoretical framing, motivation, and execution.

The central idea is to treat each vertex v as a particle located at position $\mathbf{x}(v)$ analogous to a particle in a physical system. Forces are then defined between particles corresponding to the graph topology and relative position. The specific form of the forces varies between methods, but in general they induce an energy function or dynamical system whose equilibrium configurations correspond to the layout X. Methods in this category differ primarily in three aspects:

1. *The specific form of the forces and the corresponding energy function.*
   Different models define interactions between vertices in different ways. For example, spring-electrical methods apply attractive forces along edges and repulsive forces between all vertices, while stress and strain models define spring-like forces between all vertex pairs with natural lengths proportional to their geodesic distance in G.

2. *The mathematical approach used to obtain the equilibrium configuration.*
   Equilibrium configurations may be obtained by direct physical simulation of the dynamical particle system, wherein each particle's position is iteratively updated according to the forces acting upon it, or by numerical methods that compute equilibrium states of the system, for example by solving the corresponding equilibrium equations or by minimizing the associated energy function.

3. *The computational algorithms used to calculate the resulting layout.*
   Among the many force-based layout methods that exist, many differ primarily in the implementation strategies, particularly in techniques used to improve scalability and efficiency. This is especially important for large networks, where naive implementations tend to become impractically slow and memory inefficient. Common approaches include

approximation techniques, multilevel methods, and data structures that accelerate force computation.

The general framework described above gives rise to numerous concrete layout models. In the following sections several representative approaches are presented, beginning with the classical spring–electrical model and followed by methods based on alternative energy formulations such as LinLog and Multidimensional Scaling (MDS). For each method we briefly describe the force or energy model and the algorithm used to compute the layout.

The Spring-Electrical model. This model was first introduced by Peter Eades in 1984 [18]. In this first approach, each edge in G was modeled as a spring whose attractive force grew logarithmically with distance, while non-adjacent vertices were subject to a repulsive force inversely proportional to their distance. A physical simulation was then used to find a stable layout starting from an initial configuration.

Later, Fruchterman and Reingold [19] suggested a model in which attractive spring forces are proportional to the squared distance between adjacent vertices, while repulsive electrical forces act between all vertex pairs and are inversely proportional to their distance.

Let the unit direction vector from j to i be defined as

$$\hat{\mathbf{r}}_{ij} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|} \tag{1}$$

The attractive force is

$$\mathbf{F}_a(i,j) = -\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{K}\,\hat{\mathbf{r}}_{ij} \quad i \sim j \tag{2}$$

and the repulsive force is

$$\mathbf{F}_r(i,j) = \frac{K^2}{\|\mathbf{x}_i - \mathbf{x}_j\|}\,\hat{\mathbf{r}}_{ij} \quad i \neq j \tag{3}$$

where the parameter K represents the desired edge length in the final layout.

These forces correspond to the following energy function [20]:

$$\mathcal{E}_{\mathrm{FR}}(X) = \sum_{i\sim j} \frac{1}{3K}\,\|\mathbf{x}_i - \mathbf{x}_j\|^3 - \sum_{i<j} K^2 \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{4}$$

The layout is computed iteratively. For each vertex i, the normalized force direction $\hat{\mathbf{f}}_i$ is calculated as:

$$\mathbf{F}_i = \sum_{j\sim i} \mathbf{F}_a(i,j) + \sum_{j\neq i} \mathbf{F}_r(i,j), \quad \hat{\mathbf{f}}_i = \frac{\mathbf{F}_i}{\|\mathbf{F}_i\|} \tag{5}$$

Using the previous configuration $X^{(j)}$, the vertex position is updated by moving in the direction of $\hat{\mathbf{f}}_i$ with decreasing step length $h^{(j)}$

$$\mathbf{x}_i^{(j+1)} \leftarrow \mathbf{x}_i^{(j)} + h^{(j)}\,\mathbf{f}_i \tag{6}$$

until the layout stabilizes at an equilibrium point. Later refinements introduce adaptive step-length schemes [21], [22].

Despite the physical interpretation, the algorithm does not simulate particle dynamics according to classical mechanics. Instead it behaves as a heuristic gradient-based optimization procedure for minimizing (4).

This method usually works well for small graphs, but for larger graphs it is prone to becoming trapped in local minima of the energy function [17]. In addition, the algorithm itself must calculate repulsive interactions between all $\frac{n(n-1)}{2}$ vertex pairs, resulting in $\mathcal{O}(n^2)$ computational complexity.

Fast force approximation. The time complexity can be improved by using a spatial partitioning technique such as the Barnes-Hut algorithm [23], widely used in physics for n-body simulations, which can approximate the forces in $\mathcal{O}(n \log n)$ time with good accuracy. Such an approach is taken in [24], [25] and [22].

Multilevel Methods. To address the problem of the solution becoming trapped in local minima, various algorithms **AA7:** (citations) utilize a multilevel (or multiscale) approach by generating a progressively coarser series of graphs $G = G^0, G^1, G^2, ..., G^k$, such that G^{j+1} is a coarse approximation of G^j with fewer vertices, while preserving the basic connectivity structure. Then a progressively finer series of layouts $X_k, X_{k-1}, X_{k-2}, ..., X_0 = X$ is generated, where X_j is a layout of graph G^j, using $X_j^{(0)} = X_{j+1}$ as its initial configuration. The core idea is that by having fewer vertices that encode the same basic connectivity, the energy function $\mathcal{E}^j$ will correspond to a coarse approximation of the original $\mathcal{E}^0$, while being smoother and therefore easier to traverse without becoming trapped in local minima. At the same time, since $\mathcal{E}^j$ has the same basic shape as $\mathcal{E}^{j-1}$, its minimum is likely to be close to a minimum of the finer energy function. By repeating this process in finer and finer steps, the method attempts to refine the computed equilibrium point for progressively finer versions of G. The final layout X_0 is an approximate equilibrium configuration of $\mathcal{E}^0$, but is less likely to lie in one of its many local minima compared to random initialization. Notably this approach is not limited to any specific model, or to network layout in general, but has found wide application in a variety of combinatorial optimization problems **AA8:** (more citations) .

AA9: We should consider a multi-level approach to TNE as well. A refinement strategy like the one described above could produce better results, but there is an even simpler idea: first generate a traditional vertex layout using SG-t-SNE and generate the edge layout as defined in the paper, then use the combined vertex–edge layout as the initial configuration for the TNE step. This might be able to retain some wanted properties of the vertex layout while alleviating crowding. Will test soon.

LinLog and r-PolyLog models. The Fruchterman-Reingold model tends to produce layouts with small and uniform edge lengths and few edge crossings. However, it performs poorly at separating clusters, since edges connecting different clusters should typically be longer [20].

The LinLog energy model [20] was introduced by Andreas Noack in 2004, to improve on the cluster separation problem of the Fruchterman-Reingold force model. Noack takes an energy-centric approach to defining the dynamical system of particles that represent the layout X.

Specifically the LinLog energy function $\mathcal{E}_{\text{LinLog}}$ is defined as

$$\mathcal{E}_{\text{LinLog}}(X) = \sum_{i\sim j} \|\mathbf{x}_i - \mathbf{x}_j\| - \sum_{i<j} \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{7}$$

and the layout problem is framed from the explicit perspective of finding a minimal-energy configuration X.

A specific algorithm to find the minima is not described, but standard optimization methods in conjunction with multilevel approaches may be used in practice.

LinLog has better separation of clusters with small diameter compared to Fruchterman-Reingold, but sacrifices edge length uniformity as a result.

Noack also introduces the generalized r-PolyLog class of energy models with corresponding energy functions

$$\mathcal{E}_{r\text{-PolyLog}}(X) = \sum_{i\sim j} \frac{1}{r}\, \|\mathbf{x}_i - \mathbf{x}_j\|^r - \sum_{i<j} \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{8}$$

Where parameter $r \geq 1$ describes the model behaviour. Choice $r = 1$ separates clusters while larger values of r increasingly favor uniform edge lengths.

Notice that 1-PolyLog is equivalent to LinLog (7), while the 3-PolyLog energy is the same as Fruchterman-Reingold (4), up to a constant scaling factor.

The Stress model. A limitation of Spring-Electrical models like those described above is that they primarily enforce local edge length constraints and do not explicitly encode global distance relationships between vertices. Real-world networks often include quantitative information on their edges in the form of weights representing similarity or distance between the vertices. While such information could be incorporated by adjusting the strength of attractive forces, the Stress model instead adopts a more principled approach that attempts to match the distances between all vertex pairs.

Consider a weighted graph $G = (V, E, w)$ where V and E are the vertex and edge sets, and $w : E \to W$ maps each edge e to a value $w(e) \in W$. We denote edges by $e_{ij} = (i, j)$ and write $w_{ij} = w(e_{ij})$ when $e_{ij} \in E$.

For each vertex pair (i, j) we derive a target distance $d_{ij} \in \mathbb{R}^+$. If $e_{ij} \in E$, the distance is obtained by applying a transformation $f : W \to \mathbb{R}^+$ that maps each edge weight to a distance, so that $d_{ij} = f(w_{ij})$. For non-adjacent vertices, d_{ij} is typically defined as the shortest-path distance between i and j in G when edges are assigned lengths $f(w_{ij})$. For unweighted graphs, all edges are assigned uniform length 1.

We assume that G is connected, ensuring that d_{ij} is defined for all vertex pairs. If the graph is disconnected, its connected components can be identified and the algorithm applied to each component separately.

The choice of transformation f depends on the information represented by the weights. If the weights encode distances between vertices, the identity mapping $f(w) = w$ suffices. If instead w_{ij} represents a similarity or probability, a natural choice is $f(w) = -\log(w)$. This transformation preserves the ordering of similarities and has the useful property that shortest-path distances become additive: probabilities along a path multiply while their negative logarithms

sum. Other choices are better suited to different weight models and include functions such as $f(w) = 1/w$ and $f(w) = 1 - w$.

Once the ideal distances d_{ij} are defined for all vertex pairs, the goal is to compute a layout that approximates the ideal distances as closely as possible.

In the stress model, each vertex pair is connected by a spring with nominal length d_{ij} and stiffness k_{ij}. This yields the energy function

$$\mathcal{E}_{\text{Stress}}(X) = \sum_{i<j} k_{ij} \big(\|\mathbf{x}_i - \mathbf{x}_j\| - d_{ij} \big)^2 \tag{9}$$

whose minimum corresponds to the layout that optimally matches the ideal distances. A typical choice is $k_{ij} = 1/d_{ij}^2$ which yields the equivalent formulation

$$\sum_{i<j} \left(\frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{d_{ij}} - 1 \right)^2 \tag{10}$$

thereby measuring the relative deviation between the actual and ideal edge lengths.

Although the stress model is often grouped with force-directed methods, it is more naturally interpreted as an explicit optimization formulation derived from Multidimensional Scaling (MDS) [26], [27]. It was introduced to network layout by Kamada and Kawai in 1989 [28], who proposed minimizing (9) using Newton's method. More recently, stress majorization [29] has become the preferred approach due to its robustness and guaranteed monotonic decrease of the stress function.

The Stress model as defined is not easily scalable for large networks. Constructing the model requires computing shortest-path distances for all vertex pairs. Using Johnson's algorithm this step requires $\mathcal{O}(n^2 \log n + nm)$ time and $\mathcal{O}(n^2)$ memory to store the computed distances. The stress-majorization step additionally requires solving dense linear systems in each iteration, which can be more computationally expensive than the distance computation itself. Consequently, for very large graphs the method becomes both computationally expensive and memory intensive [17]. Various optimizations and approximations have therefore been proposed that trade accuracy for reduced time and memory requirements. **AA10:** Citations?

2.1.2. Spectral Methods

Spectral layout for network visualization was introduced by Kenneth Hall in 1970 [30] and uses the spectral properties of the graph Laplacian to determine vertex positions.

In Hall's original formulation we seek to minimize the energy function

$$\mathcal{E}_{\text{Spectral}}(X) = \sum_{i \sim j} w_{ij} \, \|\mathbf{x}_i - \mathbf{x}_j\|^2 \tag{11}$$

where w_{ij} denotes the weight of edge $e_{ij} \in E$ and $w_{ij} = 0$ if $e_{ij} \notin E$. For undirected graphs $w_{ij} = w_{ji}$.

This objective penalizes large distances between adjacent vertices, weighted by their edge weights. To avoid the trivial solution $\mathbf{x}_i = \mathbf{x}_j$ for all vertices, we impose orthogonality and normalization constraints $X^T X = I$ and $X^T \mathbf{1} = 0$.

The weighted graph Laplacian is defined as

$$L^w = D - W \tag{12}$$

where D is the diagonal weighted degree matrix defined as $D_{ii} = \sum_{j \neq i} w_{ij}$ and $D_{ij} = 0$ for $i \neq j$, while W is the weighted adjacency matrix $W_{ij} = w_{ij}$ when $i \neq j$ and $W_{ii} = 0$.

Using the laplacian definition in (12), the energy (11) can be written as

$$\mathcal{E}_{\text{Spectral}}(X) = \text{tr}(X^T L^w X), \quad \text{subject to } X^T X = I \text{ and } X^T \mathbf{1} = 0 \tag{13}$$

Minimizing (13) yields solutions X^* whose columns are eigenvectors X^*_k of L^w. The energy at the optimum becomes

$$\mathcal{E}_{\text{Spectral}}(X^*) = \sum_{k=1}^{d} \lambda_k \tag{14}$$

where λ_k are the corresponding eigenvalues. For a connected graph, the smallest eigenvalue of L^w is 0 with eigenvector $\mathbf{1}/\sqrt{n}$. The constraint $X^T \mathbf{1} = 0$ excludes this trivial solution, therefore the optimal layout is obtained by selecting the d smallest non-zero eigenvalues and their corresponding eigenvectors.

The spectral layout energy function penalizes large distances between adjacent vertices, particularly those connected by strong edge weights, producing layouts that vary smoothly over the graph. Vertices with many or strong edges tend to be placed near each other, while weak connections correspond to larger separations.

However, the spectral layout method often produces less interpretable results for many complex real-world graphs. High-degree vertices tend to collapse toward the center, vertex crowding is common, and the layout may appear stretched along the eigenvector directions. Consequently, spectral layouts are rarely used as final visualizations and instead commonly serve as initial configurations for other layout algorithms.

Normally, a full eigen-decomposition requires $\mathcal{O}(n^3)$ time. However, the layout requires computing only the d smallest non-zero eigenvectors, so iterative eigensolvers can approximate the solution very efficiently even for very large graphs [31].

2.1.3. Methods based on Dimensionality Reduction

Dimensionality reduction methods have not primarily been developed in the context of network layout, but as general methods for embedding high-dimensional data into lower-dimensional spaces. However their generality also makes them useful for visualization, and specifically for network layouts.

Multidimensional Scaling (MDS), including the Stress model described in Section 2.1.1, forms a family of dimensionality reduction methods that has long been central to network layout. More recently, Stochastic Neighbor Embedding methods (SNE [14] and t-SNE [15]) have been used in network visualization [2], [32]. In addition, SNE and t-SNE, as well as more recent methods like UMAP [33] can generally be interpreted as layout methods for probabilistic neighborhood graphs derived from point-cloud data.

Stochastic Neighbor Embedding (SNE and t-SNE). Stochastic Neighbor Embedding was developed by Geoffrey Hinton and Sam Roweis in 2002 [14] as a method for general

dimensionality reduction whose objective is to preserve neighborhood relationships among the high-dimensional data points.

Given a high dimensional set of points $Y = [\mathbf{y}_1, \mathbf{y}_2, ..., \mathbf{y}_n]^T$ we attempt to find a lower dimensional layout $X = [\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n]^T$ such that the neighborhood structure is preserved.

From Y, we construct conditional neighborhood probabilities $p_{j|i}$, interpreted as the probability that point i chooses point j as a neighbor, defined as

$$p_{j|i} = \frac{\exp\big(-\|\mathbf{y}_i - \mathbf{y}_j\|^2 \ / 2\sigma_i^2\big)}{\sum_{k \neq i} \exp(-\|\mathbf{y}_i - \mathbf{y}_k\|^2 \ / 2\sigma_i^2)} \tag{15}$$

where the standard deviation σ_i is chosen for each i using binary search such that the Shannon entropy of the distribution P_i measured in bits (defined as $H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$) satisfies the equation $2^{H(P_i)} = k$. The value $2^{H(P_i)}$ is called the perplexity $\mathrm{Perp}(P_i)$, and represents the effective number of neighbors k of the distribution. The value k is chosen beforehand.

For a given low dimensional layout X we also construct a conditional distribution with *induced* conditional probability $q_{j|i}$ that point i picks point j given by

$$q_{j|i} = \frac{\exp\big(-\|\mathbf{x}_i - \mathbf{x}_j\|^2\big)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2)} \tag{16}$$

In the above formulas, the self-neighbor probabilities are excluded, so $p_{i|i} = q_{i|i} = 0$.

The objective is to make the induced neighborhood distribution in the embedding X match the target distribution derived from Y, formulated as a minimization problem of a cost function that measures the discrepancy between the neighborhood distributions in the original and embedded spaces. A natural choice is a sum of Kullback-Leibler divergences of the conditional distributions.

$$\mathcal{E}_{\mathrm{SNE}}(X) = \sum_i \mathcal{D}_{\mathrm{KL}}(P_i \parallel Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \tag{17}$$

Differentiating (17) w.r.t the vectors $\mathbf{x}_i$ results in gradient

$$\frac{\partial \mathcal{E}_{\mathrm{SNE}}}{\partial \mathbf{x}_i} = 2 \sum_{j \neq i} \big(p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j}\big)\big(\mathbf{x}_i - \mathbf{x}_j\big) \tag{18}$$

which has the interpretation of forces on $\mathbf{x}_i$, pulling toward or pushing away from $\mathbf{x}_j$, depending on whether j is assigned too much or too little neighborhood probability in the embedding.

The SNE method preserves local neighborhood structure well and can reveal cluster structure, but often suffers from substantial point crowding. Additionally, the objective (17) is difficult to optimize effectively, and the solution tends to become trapped in local minima.

Van der Maaten and Hinton later introduced t-Distributed Stochastic Neighbor Embedding (t-SNE) to address these limitations in the context of visualization. t-SNE differs from classical SNE in two major ways.

The first is the symmetrization of the neighborhood probabilities. Consider the probabilities $p_{j|i}$ and $p_{i|j}$. Generally $p_{j|i} \neq p_{i|j}$ since the standard deviations σ_i and σ_j depend on the

density of Y near points i and j. The symmetric version instead introduces a joint probability distribution P, defined by $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$, together with a corresponding low-dimensional joint distribution Q defined in the next paragraph. This allows for the objective to be written as a single KL divergence

$$\mathcal{E}_{t\text{-SNE}}(X) = \mathcal{D}_{\mathrm{KL}}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{19}$$

The definition of the joint probability Q is the second major change, as it now uses a heavy-tailed Student-t distribution instead of the previous Gaussian. The probability q_{ij} between points i and j is given by

$$q_{ij} = \frac{\left(1 + \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|\mathbf{x}_k - \mathbf{x}_l\|^2\right)^{-1}} \quad \text{for } i \neq j \tag{20}$$

For consistency, self-neighbor probabilities are again excluded in the symmetric formulation, so $p_{ii} = q_{ii} = 0$.

These two changes result in the following gradient

$$\frac{\partial \mathcal{E}_{t\text{-SNE}}}{\partial \mathbf{x}_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) \left(1 + \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)^{-1} (\mathbf{x}_i - \mathbf{x}_j) \tag{21}$$

which has a simpler symmetric form than the gradient of classical SNE.

Together, these changes mitigate both of the above limitations of SNE. The heavy-tailed kernel produces repulsive forces for dissimilar points that decay more slowly with distance compared to SNE, effectively reducing crowding in the layout. In addition, the objective (19) produces a more stable optimization landscape in practice using standard optimization methods.

t-SNE uses a modified steepest descent algorithm with momentum and an initial early exaggeration phase to optimize the energy function. In practice, this produces substantially more useful visual embeddings for many point-cloud datasets compared to classical SNE and several earlier nonlinear dimensionality reduction methods such as Sammon mapping and Isomap.

In the exact formulation of t-SNE, the joint probability distribution P is dense, since the Gaussian neighborhood model assigns nonzero probability to essentially every pair of points. However, when the perplexity is small, the vast majority of these probabilities are negligible. Consequently, practical implementations of t-SNE often replace P by a sparse approximation obtained by retaining only the strongest local neighborhood relations, typically through an exact or approximate k-nearest-neighbor search, followed by symmetrization and renormalization. In this sense, practical t-SNE is already well approximated by a sparse stochastic k-NN graph, even when the original method is formally defined on a dense similarity matrix.

To understand the computational effect of sparsifying P, it is useful to rewrite the gradient (21) using the definition of q_{ij} in (20) as

$$\frac{\partial \mathcal{E}_{t\text{-SNE}}}{\partial \mathbf{x}_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) \frac{q_{ij}}{Z} (\mathbf{x}_i - \mathbf{x}_j) \tag{22}$$

with normalization term $\frac{1}{Z} = \sum_{k \neq l} \left(1 + \|\mathbf{x}_k - \mathbf{x}_l\|^2\right)^{-1}$.

This allows us to rewrite the gradient as a sum of attractive and repulsive forces

$$\frac{\partial \mathcal{E}_{t\text{-SNE}}}{\partial \mathbf{x}_i} = \underbrace{\frac{4}{Z} \sum_{j \neq i} p_{ij} q_{ij} (\mathbf{x}_i - \mathbf{x}_j)}_{F_{\text{attr}}} - \underbrace{\frac{4}{Z} \sum_{j \neq i} q_{ij}^2 (\mathbf{x}_i - \mathbf{x}_j)}_{F_{\text{rep}}} \tag{23}$$

The sparsification substantially reduces the cost of the attractive part of the gradient, since only pairs with nonzero p_{ij} contribute. However, the repulsive part remains dense, because the low-dimensional distribution Q depends on all pairs of points. For this reason, the dominant computational cost in large-scale t-SNE lies in approximating the repulsive interactions. Barnes-Hut-SNE [34] uses a spatial tree to approximate distant repulsive forces and reduces the overall complexity to approximately $\mathcal{O}(n \log n)$, while also computing a sparse approximation of P from nearest-neighbor relations. FIt-SNE [35] accelerates the repulsive term further by interpolating the interaction kernel onto a regular grid and using the Fast Fourier Transform to evaluate the resulting convolution efficiently, together with approximate nearest-neighbor methods for constructing the sparse high-dimensional affinities.

Extension to sparse stochastic graph embedding (SG-t-SNE). The sparse formulation used in practical t-SNE suggests a natural extension to network layout. In conventional t-SNE, the target distribution P is obtained indirectly from high-dimensional feature vectors by first constructing a distance-based k-NN graph, then converting each neighborhood into a conditional Gaussian distribution, and finally symmetrizing the result. Once the target distribution P has been constructed, the subsequent embedding optimization depends only on P and the induced low-dimensional distribution Q, not directly on the original point-cloud coordinates Y.

This suggests that the probability-matching formulation of t-SNE need not be tied specifically to point-cloud data. Instead, one may ask whether the same objective can be posed directly on a sparse stochastic graph given as input, rather than using such a graph only as an approximation to a dense similarity model. SG-t-SNE [2] removes this interface restriction and instead takes as input a sparse stochastic graph directly.

The method takes as input a sparse stochastic graph $G = (V, E, P_c)$, where $P_c = \left[p_{j|i}\right]$ is a row-stochastic conditional probability matrix supported on the adjacency pattern of G.

In the formulation used in practice, SG-t-SNE applies a row-wise nonlinear rescaling to P_c, constructing a parameterized family of stochastic matrices $P_c^{(\lambda)} = \left[p_{j|i}^{(\lambda)}\right]$. For each vertex i, an exponent γ_i is chosen such that

$$\sum_{j \sim i} p_{j|i}^{\gamma_i} = \lambda \tag{24}$$

and the rescaled conditional probabilities are then defined, such that each row remains stochastic. They are given by

$$p_{j|i}^{(\lambda)} = \frac{p_{j|i}^{\gamma_i}}{\lambda} \tag{25}$$

This corresponds to the special case used in the reported experiments, where the more general monotone transform of the original formulation is taken to be the identity.

This rescaling plays a role analogous to the perplexity-based bandwidth selection used in SNE and t-SNE, but is adapted directly to a sparse stochastic graph. Rather than deriving neighborhood distributions from distances and then enforcing a fixed entropy level, it allows the concentration of the existing conditional probabilities to be varied directly while preserving the sparsity pattern of the graph.

As in t-SNE, this rescaled conditional model is symmetrized into a joint target distribution

$$P^{(\lambda)} = \frac{P_c^{(\lambda)} + \left(P_c^{(\lambda)}\right)^T}{2n} \tag{26}$$

while the low-dimensional distribution Q is still defined by the Student-t kernel in the layout space, like in (20). The embedding is then obtained by minimizing the same Kullback-Leibler divergence $\mathcal{D}_{\mathrm{KL}}\big(P^{(\lambda)} \| \, Q\big)$ as in t-SNE.

By varying λ, the method interpolates between more concentrated and more uniform neighborhood distributions on the fixed sparse support of the graph, thereby providing a controllable family of target probability models for the embedding.

Although the input graph is sparse, the low-dimensional distribution Q remains dense, so the repulsive term in the gradient still involves all pairs of vertices. For this reason, the practical implementation SG-t-SNE-Π uses t-SNE-Π as its computational back-end. t-SNE-Π is a high-performance computational implementation for large sparse graph embeddings, accelerating both the sparse attractive term and the dense repulsive term through permutations and memory-aware access patterns tailored to modern hierarchical memory architectures.

SG-t-SNE-Π additionally enables efficient computation of 3D embeddings, something not previously possible even for the accelerated t-SNE methods like BH-t-SNE and FIt-SNE. The higher dimensionality enables the embedding to capture more complex structural relationships in the data.

However, SG-t-SNE still suffers from some of the limitations of SNE based methods. Embeddings often include substantial vertex crowding and excessive edge crossing.

NP1: Please check that everything here is correct

2.2. State of the Art

Various software packages exist for the practical embedding of networks.

We briefly present a set of practical network layout algorithms based on the principles described in Section 2.1. The selected methods were chosen to represent a broad range of layout paradigms in terms of mathematical formulation, layout objective, and computational scope.

These algorithms will later be used in the experimental comparison with our method TWIN.

Fruchterman-Reingold. The classical Fruchterman-Reingold (FR) method [36] is a force-directed spring-electrical layout algorithm described in Section 2.1. It seeks layouts with

approximately uniform edge lengths while maintaining global vertex separation through repulsive forces. We use the *NetworkX* [37] implementation via the `spring_layout` method.

Yifan Hu. The Yifan Hu method [22] is a scalable force-directed layout algorithm that combines a multilevel strategy with Barnes-Hut approximation of repulsive forces. It belongs to the spring-electrical family, but is designed specifically for efficient layout of large graphs. We use the *GraphViz* [38] implementation `sfdp`.

ForceAtlas2. ForceAtlas2 [39] is a force-directed layout method with a force model that differs from classical FR and is designed for interactive exploratory network visualization. It is the default layout algorithm in the widely used network visualization software *Gephi* [40], and often produces visually effective general-purpose layouts across a broad range of networks. Its optional "LinLog" mode is inspired by Noack's LinLog model and tends to improve cluster separation, although it is not a direct implementation of the original LinLog energy. We use the implementation in the `fa2` Python package.

Kamada-Kawai. The classical Kamada-Kawai method [28] is a stress-based layout method that aims to preserve graph-theoretic distances between vertex pairs. It is generally best suited to small and medium-sized networks, where the all-pairs distance model remains computationally practical. We use the *NetworkX* [37] implementation via the `kamada_kawai_layout` method.

Stress Majorization (MDS-style). The *GraphViz* [38] engine `neato` implements a stress-based layout approach closely related to multidimensional scaling, using stress majorization [29] for more stable optimization of the stress objective. Compared to classical Kamada-Kawai, this typically yields more robust convergence behavior and better practical layouts for larger graphs.

Spectral Layout. Spectral layout, as described in Section 2.1, uses eigenvectors of the graph Laplacian to embed the graph in Euclidean space. It emphasizes adjacency preservation and tends to produce smooth, globally structured layouts, but does not explicitly optimize a force or stress objective. We use a custom Python implementation which computes the d dominant eigenvectors of the graph Laplacian.

SG-t-SNE-Π. This method is described in detail in Section 2.1. It is a stochastic graph embedding method based on the t-SNE probability-matching formulation, using a sparse graph-derived target distribution together with Student-t low-dimensional affinities. We use the `sgtsnepi` Python package, which wraps the reference SG-t-SNE-Π implementation.

AA11: I will create a table summary for each method comparing Formulation, Primary objective, Local neighborhood preservation, Global structure preservation, Cluster separation tendency, Scalability, and Implementation used

3. Layout evaluation and metrics

As discussed in the previous section, a given graph may correspond to infinitely many layout configurations. However, these configurations are not equally useful. Although this is obvious

at an abstract level, it is difficult to define concretely what layout quality means in the context of network visualization.

In scientific network analysis, a layout is useful insofar as it improves our understanding of the underlying network. In general, this means that the geometry of the layout should reflect structural relationships present in the graph. Vertices, edges, communities, paths, weights, and other network features should be represented in a way that supports visual interpretation.

The difficulty is that any visually interpretable layout exists in a low-dimensional space, and therefore cannot preserve the full complexity of a general network. A layout can only emphasize a subset of the network structure. Moreover, preserving one structural aspect may obscure another, so every layout method makes trade-offs in what it represents.[2] Consequently, there is no single universal measure of layout quality. Instead, a layout should be evaluated according to the degree to which it preserves or reveals specific aspects of the network structure. The purpose of the following section is therefore to define a set of evaluation criteria and corresponding metrics, each measuring a distinct property relevant to the goals of this thesis.

These criteria must connect graph-theoretic structure to geometric structure: we must specify which properties of the network should be preserved, and what spatial relationships in the layout are taken to represent them.

In this context, the relative geometry of the layout is central to visual interpretation. In particular, most of the metrics considered below depend on relative distances between points, since these determine proximity, separation, crowding, neighborhood structure, and edge length. Additionally, the absolute coordinate system of a network layout has no intrinsic meaning. Two coordinate representations that differ only by translation, rotation, reflection, or uniform scaling are visually equivalent for the purposes of this thesis.

In practice, two complementary modes of layout evaluation are commonly used.

1. The first is visual examination. A layout is computed for a graph with known structural properties, and the result is inspected to determine whether those properties are visually apparent. For example, adjacent vertices should often appear in close proximity, communities should be visually distinguishable, and graph-theoretic relationships should be reflected in the spatial organization of the drawing. This approach has the advantage of directly evaluating the layout as a visual object, without requiring formal metric definitions. However, it is also inherently subjective, since the interpretation of a layout depends on the observer's experience, expectations, and visual judgement.
2. The second mode of evaluation is quantitative. One constructs a set of numerical measures, or *metrics*, that attempt to quantify specific aspects of structure preservation by comparing geometric properties of the layout with structural properties of the underlying graph. Such metrics are useful for comparing layout algorithms in a reproducible way, since they provide concrete numerical scores. However, they should not be treated as definitive measures of layout quality. The choice of metric is itself a modeling decision, and different metrics may measure different aspects of the same informal criterion. Moreover, a favorable metric value does not necessarily imply that a layout is visually or analytically useful in practice.

[2] We have already seen an example of such a trade-off in Section 2.1.1 between the Fruchterman-Reingold and LinLog force models: preserving proximity of adjacent vertices and uniformity of edge lengths can inhibit community separation, and vice-versa. The r-PolyLog model makes this trade-off explicit.

For this reason, the evaluation in this thesis uses both visual examination and quantitative metrics. Visual results are used to assess whether the structural features of a graph are interpretable in the drawing, while quantitative metrics are used to measure specific properties such as neighborhood preservation, class separability, point crowding, edge geometry, and global distance preservation. The remainder of this section defines the metrics used in the evaluation and clarifies which aspect of layout quality each metric is intended to measure.

3.1. Evaluation Metrics

As previously described, it is useful to construct numerical measures aimed at quantitatively evaluating the quality of a graph layout.

The measures considered here correspond broadly to two categories of desirable properties in scientific network layout: (i) the degree to which structural features of the graph are preserved in the layout and (ii) the degree to which the layout is visually clear and free from artifacts such as crowding, clutter, or excessive edge complexity.

In what follows, we use criterion to refer to the qualitative property of interest, such as neighborhood preservation or point crowding, and metric to refer to the concrete numerical quantity used to evaluate that property.[3]

Not every structural feature of a graph has a canonical geometric equivalent in a layout. A graph and a set of points in Euclidean space are different mathematical objects, and a layout algorithm necessarily chooses how graph relationships are to be represented geometrically. Consequently, every criterion requires a modeling decision: one must decide which graph-theoretic property is being evaluated and which geometric property is taken to represent it.

For example, suppose we want the adjacency relation between two vertices $i, j \in V$ to be preserved by their point representations $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^d$. Since geometric points cannot be adjacent in the graph-theoretic sense, adjacency must be translated into some geometric criterion. One natural interpretation is proximity: adjacent vertices should be placed close to one another relative to non-adjacent vertices. This can be measured in several ways. One may compare the distances of adjacent and non-adjacent vertex pairs, or construct the k nearest neighbors of each point in the layout and compare them to the graph neighborhood of the corresponding vertex

Each of these choices captures a different interpretation of the same informal criterion. There is therefore no canonical metric for a general property such as “adjacency preservation” or “visual clarity.” For this reason, the evaluation in this thesis adopts a pluralistic approach: for each structural or visual property of interest, we use one or more representative metrics that capture common and practically useful interpretations of that property.

3.1.1. Structural Feature Preservation

Consider an (optionally weighted) graph $G = (V, E, w)$ with n vertices and m edges, and a layout $X \in \mathbb{R}^{n \times d}$. A layout metric is a rule f which assigns a numerical value to such a pair (G, X).

For a fixed graph G, we write the corresponding graph-specific metric as $X \mapsto f(X; G)$, where the notation emphasizes that the layout X is the object being evaluated while the graph G provides the structural reference.

[3]This use of the word metric should not be confused with a metric as a distance function on a metric space.

Some metrics are first defined locally, for each vertex or edge, and then aggregated over the graph. In such cases $f(X;G)$ denotes the final aggregate score unless otherwise stated.

For weighted graphs we define $w_{ij} = w(e_{ij})$ to be the weight of edge $e_{ij} \in E$, and $w_{ij} = 0$ when $e_{ij} \notin E$. For unweighted graphs, we construct the trivially weighted graph where all weights are 1, such that $w_{ij} = 1$ if $i \sim j$, and $w_{ij} = 0$ otherwise. In this case, the weights are taken to represent a sense of similarity, or affinity, rather than distance. This means that vertices which are connected with high edge weights are more closely related, something which is often desirable to preserve in the layout.

Conversely, we may define an ideal distance d_{ij} between each pair of vertices. Similar to the definition in Section 2.1.1, we generally define d_{ij} for adjacent vertices to be a function of the weight of the edge $f(w_{ij})$, and for non-adjacent vertices to be the shortest-path length between i and j induced by these edge lengths.

Datasets of networks may provide values for the edges corresponding to either similarities/ strengths/affinity scores or to distances/costs. Since these interpretations are inversely ordered but not metrically equivalent, any conversion from affinity to distance requires an explicit modeling choice. At minimum the transformation should be monotone decreasing. Stronger requirements and the specifics of this conversion are outside the scope of this thesis, however, we mention that some common choices used in practice include $d = 1/w$, $d = 1 - w$ and $d = -\log(w)$.

When a local score is undefined because its denominator is zero, the corresponding vertex is excluded from the aggregate unless otherwise stated. For metrics based on graph-theoretic distances, we evaluate only vertex pairs with finite distance, or equivalently restrict the metric to connected components according to the convention specified in the experimental setup.

Adjacency Distance Ratio. We would like to preserve the adjacency relationship between vertices in G in the layout X. As previously described, this can be done by measuring the proximity of adjacent or strongly related vertices relative to the overall spatial scale of the layout. There are several possible interpretations of this criterion; following our pluralistic approach, we choose one representative metric.

We measure proximity-based adjacency preservation by comparing the average embedded distance of graph-related vertex pairs against the average embedded distance of all vertex pairs. For a weighted graph $G = (V, E, w)$, we define

$$\mathrm{ADR}(X;G) = \frac{\frac{1}{W}\sum_{i<j} w_{ij}\ \|\mathbf{x}_i - \mathbf{x}_j\|}{\frac{2}{n(n-1)}\sum_{i<j}\|\mathbf{x}_i - \mathbf{x}_j\|} \tag{27}$$

where

$$W = \sum_{i<j} w_{ij} \tag{28}$$

Smaller values indicate that strongly related vertices are placed closer together relative to the global spatial scale of the layout. Typically, we want $\mathrm{ADR}(X;G) < 1$.

For an unweighted graph, this reduces to

$$\mathrm{ADR}(X;G) = \frac{\frac{1}{m}\sum_{i\sim j}\|\mathbf{x}_i - \mathbf{x}_j\|}{\frac{2}{n(n-1)}\sum_{i<j}\|\mathbf{x}_i - \mathbf{x}_j\|} \tag{29}$$

This metric is not intended to exhaust all possible interpretations of adjacency preservation. It captures one simple and practically useful interpretation: graph-related pairs should be short relative to the global spatial scale of the layout. Neighborhood-recovery metrics based on k-nearest neighbors, discussed next, capture the complementary rank-based question of whether graph-neighbors are recovered among spatial neighbors.

***k*-NN Neighborhood Recovery.** This metric corresponds to a rank-based interpretation of the same criterion: if adjacency is preserved by spatial proximity, then graph-neighbors of a vertex should appear among its nearest embedded neighbors. In contrast to the previous distance-ratio metric, this metric does not compare distance values directly, but instead evaluates the ordering induced by embedded distances.

Consider the vertex $i \in V$. We define its graph neighborhood as $\mathcal{N}_G(i) = \{j \in V \mid i \sim j\}$ and its degree $\deg(i) = |\mathcal{N}_G(i)|$. For the point $\mathbf{x}_i \in X$ we find its k-NN neighborhood defined as

$$\mathcal{N}_X^k(i) = \big\{j \in V \mid j \neq i \text{ and } \mathbf{x}_j \text{ is among the } k\text{-nearest neighbors of } \mathbf{x}_i \text{ in } X\big\} \tag{30}$$

The vertices $j \in \mathcal{N}_G(i) \cap \mathcal{N}_X^k(i)$ are called true positives, since they denote true neighbors of i recovered from the k nearest neighbors of $\mathbf{x}_i$. We would like then, to maximize the number of true positives $\mathrm{TP}_i = |\mathcal{N}_G(i) \cap \mathcal{N}_X^k(i)|$. However, the absolute number of true positives is not by itself an appropriate metric. The maximum possible value of TP_i depends on both the degree of i and the chosen neighborhood size k, since

$$\mathrm{TP}_i \leq \min\{\deg(i), k\} \tag{31}$$

As a result, vertices of large degree may contribute more true positives simply because they have more graph-neighbors that can be recovered. A global sum of true positives would therefore be dominated by high-degree vertices, rather than measuring neighborhood recovery in a vertex-balanced way. For this reason, we normalize the overlap using standard set-recovery scores.

- The **Recall** of the graph neighborhood of i is defined as

$$R_i^k(X;G) = \frac{|\mathcal{N}_G(i) \cap \mathcal{N}_X^k(i)|}{|\mathcal{N}_G(i)|} = \frac{\mathrm{TP}_i}{\deg(i)} \tag{32}$$

  and measures the fraction of true graph-neighbors of i that are recovered among its k nearest embedded neighbors.

- The **Precision** of the recovered embedded neighborhood is defined as

$$P_i^k(X;G) = \frac{|\mathcal{N}_G(i) \cap \mathcal{N}_X^k(i)|}{|\mathcal{N}_X^k(i)|} = \frac{\mathrm{TP}_i}{k} \tag{33}$$

  and measures the fraction of the k nearest embedded neighbors of i that are true graph-neighbors.

- Finally, the **Jaccard index** is defined as

$$J_i^k(X;G) = \frac{|\mathcal{N}_G(i) \cap \mathcal{N}_X^k(i)|}{|\mathcal{N}_G(i) \cup \mathcal{N}_X^k(i)|} = \frac{\mathrm{TP}_i}{\deg(i) + k - \mathrm{TP}_i} \tag{34}$$

and measures the relative overlap between the graph neighborhood and the embedded neighborhood, penalizing both graph-neighbors that are not recovered and embedded neighbors that are not graph-neighbors.

Then the associated global scores become

$$M^k(X;G) = \operatorname*{mean}_{i \in V} \big\{ M_i^k(X;G) \big\} \tag{35}$$

with $M \in \{R, P, J\}$, possibly excluding zero-degree vertices for Recall, since that would result in an undefined result.

Since the graph-neighborhood sizes are given by the vertex degrees, the choice of k should be interpreted relative to the degree distribution of the graph. When k is fixed globally, the different normalized scores have different degree-dependent ceilings. Precision can only reach 1 for vertices with $\deg(i) \geq k$, while recall can only reach 1 for vertices with $\deg(i) \leq k$. Therefore, fixed-k precision tends to favor high-degree vertices, whereas fixed-k recall tends to favor low-degree vertices. The Jaccard index balances the two effects and is largest when the graph degree of the vertex is comparable to k. For this reason, it is often useful to evaluate the scores for several values of k, rather than relying on a single neighborhood scale.

Weighted k-NN Neighborhood Recovery. The previous metric treats neighborhood recovery combinatorially: each graph neighbor is either recovered or not recovered. For weighted graphs, this may not capture the intended notion of neighborhood preservation. For example, a weighted graph may be fully connected from a combinatorial point of view, while most of the weight of each vertex is concentrated on only a small number of strong relations. In this case, recovering a weakly related vertex and recovering a strongly related vertex should not contribute equally.

One possible solution is to sparsify the weighted graph and then apply the combinatorial metric. For example, for each vertex one could retain the strongest edges whose cumulative weight exceeds a prescribed threshold.

However, a more direct approach is to measure how much of the vertex's incident weight is recovered among its nearest embedded neighbors.

$$W_i^k(X;G) = \frac{\sum_{j \in \mathcal{N}_X^k(i)} w_{ij}}{\sum_{j \neq i} w_{ij}} \tag{36}$$

This measures the fraction of the total weight of i that is recovered among the k nearest embedded neighbors of $\mathbf{x}_i$.

Equivalently, if the normalized weights

$$p_{ij} = \frac{w_{ij}}{\sum_{j \neq i} w_{ij}} \tag{37}$$

are interpreted as a probability distribution over vertices adjacent or related to i, then $W_i^{k(X;G)}$ is the probability mass recovered by the embedded k-nearest-neighbor set.

The associated global metric is then given as

$$W^k(X;G) = \underset{i\in V}{\text{mean}}\{W_i^k(X;G)\} \tag{38}$$

For an unweighted graph, with $w_{ij} = 1$ when $i \sim j$ and $w_{ij} = 0$ otherwise, this definition reduces to combinatorial recall:

$$\begin{aligned} W_i^k(X;G) &= \frac{\sum_{j\in\mathcal{N}_X^k(i)} w_{ij}}{\sum_{j\neq i} w_{ij}} = \frac{\sum_{j\in\mathcal{N}_X^k(i)\cap\mathcal{N}_G(i)} 1}{\sum_{j\in\mathcal{N}_G(i)} 1} \\ &= \frac{|\mathcal{N}_G(i)\cap\mathcal{N}_X^k(i)|}{|\mathcal{N}_G(i)|} = \frac{\text{TP}_i}{\deg(i)} = R_i^k(X;G) \end{aligned} \tag{39}$$

For this reason, we refer to this metric as weighted recall.

Scale-Normalized Stress. Another graph-level feature we would like preserved is the global distances between vertices. For a graph G we can derive a set of ideal distances d_{ij} for every vertex pair. We say that a layout X is global distance preserving, if the geometric distances between points in X are proportional to the distances in d_{ij}. Since it is generally impossible to exactly realize the graph distances as Euclidean distances in $\mathbb{R}^d$, we must construct a metric which measures the degree of distance preservation.

Borrowing from MDS and the Stress model defined in Section 2.1.1, we can use the Raw Stress function as a crude metric for global distance deviation

$$\text{Stress}(X;G) = \sqrt{\sum_{i<j}\left(d_{ij} - \|\mathbf{x}_i - \mathbf{x}_j\|\right)^2} \tag{40}$$

This function does indeed penalize non-matching distances between d_{ij} and $\|\mathbf{x}_i - \mathbf{x}_j\|$, however it is too strict in that sense. There are two main issues with using Raw Stress as a metric, and they both stem from its non-invariance to scale:

1. It is sensitive to the overall scale of the distances d_{ij}, so the magnitude of the score reflects not only distortion but also the absolute scale of the chosen ideal-distance model.
2. More importantly, it is not invariant to scaling transformations of the layout $X \mapsto \beta X$. In fact, it assumes that X has the same implicit scale as the distances d_{ij} which cannot be guaranteed for every layout algorithm. As a result, Raw Stress is not an appropriate metric for comparing across different layout algorithms.

A common improvement over Raw Stress is to normalize the squared error. There are several related normalizations in the literature. In Multidimensional scaling, Kruskal's Stress-1 [27] normalizes the squared error by $\sum_{i<j}\|\mathbf{x}_i - \mathbf{x}_j\|^2$, while another common form, often also called normalized stress, normalizes by $\sum_{i<j} d_{ij}^2$. In our notation, the latter is

$$\text{NStress}(X;G) = \sqrt{\frac{\sum_{i<j}\left(d_{ij} - \|\mathbf{x}_i - \mathbf{x}_j\|\right)^2}{\sum_{i<j} d_{ij}^2}} \tag{41}$$

This normalization makes the score dimensionless and expresses the squared error relative to the total squared magnitude of the target distance matrix. However, it does not fully solve the scale issue. If the layout is uniformly rescaled as $X \mapsto \beta X$, then

$$\mathrm{NStress}(\beta X; G) = \sqrt{\frac{\sum_{i<j} \left(d_{ij} - \beta \|\mathbf{x}_i - \mathbf{x}_j\|\right)^2}{\sum_{i<j} d_{ij}^2}} \tag{42}$$

which is generally not equal to $\mathrm{NStress}(X; G)$. Thus, two layouts that differ only by a uniform scaling transformation may receive different scores.

Kruskal's Stress-1 partly alleviates this issue by normalizing with respect to the squared layout distances, but it is still not invariant under uniform rescaling of the layout. Since the global scale of a layout is arbitrary, this remaining scale dependence is undesirable when comparing different layout algorithms.

For this reason, we use scale normalized stress [41], [42]. Because the distances d_{ij} define the target graph geometry, we keep the target-distance normalization, but first rescale the embedded distances by the optimal scalar factor $\alpha > 0$:

$$\begin{aligned} \mathrm{SNS}(X; G) &= \min_{\alpha>0} \mathrm{NStress}(\alpha X; G) \\ &= \min_{\alpha>0} \sqrt{\frac{\sum_{i<j} \left(d_{ij} - \alpha \|\mathbf{x}_i - \mathbf{x}_j\|\right)^2}{\sum_{i<j} d_{ij}^2}} \end{aligned} \tag{43}$$

The optimal value for α can be found analytically and turns out to be

$$\alpha^* = \frac{\sum_{i<j} d_{ij} \, \|\mathbf{x}_i - \mathbf{x}_j\|}{\sum_{i<j} \|\mathbf{x}_i - \mathbf{x}_j\|^2} \tag{44}$$

Therefore, the metric used in the evaluation is

$$\mathrm{SNS}(X; G) = \sqrt{\frac{\sum_{i<j} \left(d_{ij} - \alpha^* \|\mathbf{x}_i - \mathbf{x}_j\|\right)^2}{\sum_{i<j} d_{ij}^2}} \tag{45}$$

This metric measures the discrepancy between graph distances and embedded distances after accounting for the arbitrary global scale of the layout. It can be proven that it is invariant to scaling transformations of both the layout X and the ideal distances d_{ij}. The square root is included so that the stress value is expressed on the scale of relative distance error rather than squared relative error; it does not affect the optimal scaling factor. Smaller values of SNS indicate better global distance preservation.

3.1.2. Label-based Feature Preservation

Many real-world networks include labels or categories associated with their vertices. These labels may correspond to known communities, classes, topics, functional groups, or other external attributes of the entities represented by the graph. When such labels are meaningful, a useful layout should often make them visually interpretable: vertices with the same label should tend to form compact and visually distinguishable regions in the drawing.

Community labels are an important special case. Although community structure is difficult to define uniquely, communities are generally understood as groups of vertices which are more densely connected internally than externally. Community detection methods attempt to infer such groups from the graph topology by assigning to each vertex i a label ℓ_i, such that each community is the set of vertices with a common label:

$$C(\ell) = \{i \in V \mid \ell_i = \ell\} \tag{46}$$

However, labels used for evaluation need not be produced by a community detection algorithm. In many datasets, labels are supplied externally and correspond to attributes of the represented entities. For example, in a citation network where each vertex represents an academic article, labels may indicate the article's author, publication venue, research topic, or field. Such labels are not guaranteed to be reflected in the graph topology, but in many real-world networks they correlate with meaningful structural organization.

There is therefore a methodological question about which reference partition should be used when evaluating label or community preservation in a layout. Using external labels may evaluate layout algorithms according to information which is not necessarily encoded in the graph structure available to them. On the other hand, using labels produced by a community detection algorithm imposes the assumptions of that algorithm onto the evaluation. These assumptions may differ significantly between algorithms, and may even overlap with the machinery used by some layout methods. In such cases, the evaluation may favor layouts which agree with the assumptions of the detection algorithm rather than layouts which more generally reveal meaningful structure.

For this reason, the purpose of the following metrics is not to measure agreement with a particular community detection algorithm, but to assess whether semantically meaningful labels present in the data are reflected in the visualization. We therefore use external labels when available. Although such labels are not guaranteed to correspond to structural communities in the graph, they are independent of both the layout and evaluation procedures, and thus provide a common reference against which different layout algorithms may be compared.

Whenever a metric uses external labels, we also compute the corresponding graph-based score when possible. This allows layout results to be interpreted relative to the extent to which the same labels are encoded in the graph itself. Thus, we compare the graph and the layout according to their ability to reflect the same external partition under the same evaluation criterion.

Silhouette Score. For a given layout, in order for labeled groups to be visually distinct, vertices with the same label should be placed compactly in the layout, with good relative separation between groups, thus creating visually distinguishable point clusters.

One metric to measure this criterion is the commonly used silhouette score. It is defined for a single vertex as the normalized difference of distances between the points in its own label group and the points in the closest neighboring group.

Let $\mathcal{L}$ be the set of unique labels such that for every $i \in V \Rightarrow \ell_i \in \mathcal{L}$. Then, let $\mathcal{C}$ be the set of all labeled communities:

$$\mathcal{C} = \{C(\ell) \mid \ell \in \mathcal{L}\} \tag{47}$$

We define the label group of vertex i as

$$C_i = C(\ell_i) = \{j \in V \mid \ell_j = \ell_i\}. \tag{48}$$

We calculate its average distance to points in its own community as

$$a_i = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} \|\mathbf{x}_j - \mathbf{x}_i\| \tag{49}$$

Then the average distance to points in the closest neighboring community is

$$b_i = \min_{C \in \mathcal{C}, C \neq C_i} \frac{1}{|C|} \sum_{j \in C} \|\mathbf{x}_j - \mathbf{x}_i\| \tag{50}$$

The silhouette score for vertex i is defined as

$$\mathrm{Sil}_i(X; \ell) = \frac{b_i - a_i}{\max\{a_i, b_i\}} \tag{51}$$

This value ranges for each vertex from -1 to $+1$, where large positive values indicate a vertex is well matched to its own community, large negative values indicate that the vertex is closer, on average, to another community than to its assigned community, and values near 0 indicate that the vertex is close to the border between the communities. Notably, this score is not well defined for singleton communities where $|C_i| = 1$, so in our implementation we set the score for these vertices as 0.

The global score is given as the mean of the scores for each point

$$\mathrm{Sil}(X; \ell) = \frac{1}{n} \sum_{i \in V} \mathrm{Sil}_i(X; \ell) \tag{52}$$

A large positive global score indicates compact and well-separated labeled communities in the layout. Conversely, negative scores indicate that vertices are, on average, closer to other labeled groups than to their assigned group.

As mentioned, we can use a similar metric in order to test whether the external labels reflect the graph topology: given the external labels we define the graph-theoretic silhouette score given by

$$a_i^* = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d_{ij} \tag{53}$$

$$b_i^* = \min_{C \in \mathcal{C}, C \neq C_i} \frac{1}{|C|} \sum_{j \in C} d_{ij} \tag{54}$$

$$\mathrm{Sil}_i^*(G; \ell) = \frac{b_i^* - a_i^*}{\max\{a_i^*, b_i^*\}} \tag{55}$$

Here d_{ij} denotes the graph-theoretic distances like described previously. Then the global score $\mathrm{Sil}^*(G; \ell)$ is defined as above.

***k*-NN Classification Accuracy.** Another metric corresponding to the same criterion is to measure the accuracy of using a k-NN based classifier in its recovery of the initial labels. In contrast to Silhouette score, which is a global score of separation and compactness, this metric measures instead local label-homogeneity. This is based on the principle that a label-preserving layout should place each vertex near other vertices with the same label. Therefore, predicting the label of a vertex from the labels of its nearest neighbors should be accurate.

A classifier is defined as a procedure which assigns each vertex i to a predicted label $\hat{\ell}_i$, without prior knowledge of ℓ_i. A classifier of the form we discuss, however, has access to the labels ℓ_j with $j \neq i$.

Specifically, let $\mathcal{N}_X^k(i)$ denote the set of k nearest neighbors of x_i in the layout X. Equivalently, this defines the k-NN graph G_X^k of X, whose vertices are the vertices of G, and where each vertex i is connected to its k nearest neighbors in X. Notably, this graph is not necessarily undirected, since there is no guarantee that

$$j \in \mathcal{N}_X^k(i) \Leftrightarrow i \in \mathcal{N}_X^k(j) \tag{56}$$

For each vertex i, we construct a score for each possible label $\ell \in \mathcal{L}$ according to the labels of the embedded neighbors of i. The simplest form of this is a voting scheme, where the score is the number of neighbors that have label ℓ:

$$s_i(\ell) = |\mathcal{N}_X^k(i) \cap C(\ell)| \tag{57}$$

Alternatively, we may weigh the score according to the distance of $\mathbf{x}_j$ to $\mathbf{x}_i$, such that closer points contribute more strongly than those farther away:

$$s_i(\ell) = \sum_{j \in \mathcal{N}_X^k(i) \cap C(\ell)} f\big(\|\mathbf{x}_i - \mathbf{x}_j\|\big) \tag{58}$$

Here f is a decreasing function of distance, so that larger distances correspond to smaller contributions. The choice of $f(d) = 1$ reduces to the simple voting scheme, while a common distance-weighted alternative is $f(d) = 1/d$.

In all cases, the classifier is defined by selecting the label with the highest score:

$$\hat{\ell}_i = \operatorname*{argmax}_{\ell \in \mathcal{L}} s_i(\ell) \tag{59}$$

The k-NN classification accuracy is then defined as the fraction of vertices whose predicted labels match their true labels.

$$\mathrm{Acc}^k(X; \ell) = \frac{1}{n} \sum_{i \in V} \big[\hat{\ell}_i = \ell_i\big] \tag{60}$$

Here $[\cdot]$ denotes the Iverson bracket. A high value of $\mathrm{Acc}^k(X; \ell)$ indicates that the local neighborhoods in the layout are label-homogeneous enough to recover the original labels from nearby vertices.

Conversely, the equivalent graph metric uses a neighborhood classifier which assigns a label to each vertex according to the labels of its 1-hop neighborhood in the graph. The voting scheme from above becomes:

$$s_i^*(\ell) = |\mathcal{N}_G(i) \cap C(\ell)| \tag{61}$$

For weighted graphs, we may instead weigh the score according to the edge weights:

$$s_i^*(\ell) = \sum_{j \in \mathcal{N}_G(i) \cap C(\ell)} w_{ij} \tag{62}$$

Then as above, the graph-based predicted labels are

$$\hat{\ell}_i^* = \operatorname*{argmax}_{\ell} s_i^*(\ell) \tag{63}$$

The corresponding graph-neighborhood classification accuracy is

$$\mathrm{Acc}^*(G;\ell) = \frac{1}{n}\sum_i \left[\hat{\ell}_i^* = \ell_i\right] \tag{64}$$

For isolated vertices, the graph neighborhood contains no information from which the label can be recovered, so such vertices are excluded from this graph-based baseline.

This graph-based score measures how strongly the external labels are reflected in the one-hop topology of the graph. It therefore provides a baseline for interpreting the layout-based classification accuracy: a low layout score is less meaningful when the same labels are weakly recoverable from the graph itself.

3.1.3. Evaluation of Visual Quality

4. The Twin Neighbor Embedding method

AA12: The method, how and why it works.

5. Results

AA13: Including visualization results as well as computed metrics

6. The TWIN software package and GUI

AA14: Brief description of the Software, the parameters and the GUI

7. Conclusion and Limitations

AA15: What the method improves on, where it doesn't, and how it may be improved in the future.

Bibliography

[1] A. Athanasiadis, D. Floros, N. Pitsianis, and X. Sun, "Twin Neighbor Embedding of Sparse Data Networks," 2025.

[2] N. Pitsianis, A.-S. Iliopoulos, D. Floros, and X. Sun, "Spaceland Embedding of Sparse Stochastic Graphs," in *IEEE HPEC*, 2019, pp. 1–8. doi: 10.1109/HPEC.2019.8916505.

[3] K. Melnyk, K. Weimann, and T. O. F. Conrad, "Understanding Microbiome Dynamics via Interpretable Graph Representation Learning," *Scientific Reports*, vol. 13, no. 1, p. 2058, Feb. 2023, doi: 10.1038/s41598-023-29098-7.

[4] H. Ashoor *et al.*, "Graph Embedding and Unsupervised Learning Predict Genomic Sub-Compartments from HiC Chromatin Interaction Data," *Nature Communications*, vol. 11, no. 1, p. 1173, Mar. 2020, doi: 10.1038/s41467-020-14974-x.

[5] F. Costa, D. Grün, and R. Backofen, "GraphDDP: A Graph-Embedding Approach to Detect Differentiation Pathways in Single-Cell-Data Using Prior Class Knowledge," *Nature Communications*, vol. 9, no. 1, p. 3685, Sept. 2018, doi: 10.1038/s41467-018-05988-7.

[6] U. Stelzl *et al.*, "A Human Protein-Protein Interaction Network: A Resource for Annotating the Proteome," *Cell*, vol. 122, no. 6, pp. 957–968, Sept. 2005, doi: 10.1016/j.cell.2005.08.029.

[7] H.-J. Park and K. Friston, "Structural and Functional Brain Networks: From Connections to Cognition," *Science*, vol. 342, no. 6158, p. 1238411, Nov. 2013, doi: 10.1126/science.1238411.

[8] C. Kadushin, Ed., *Understanding Social Networks*. New York: Oxford University Press, 2012.

[9] V. Filipov, A. Arleo, and S. Miksch, "Are We There yet? A Roadmap of Network Visualization from Surveys to Task Taxonomies," *Computer Graphics Forum*, vol. 42, no. 6, p. e14794, 2023, doi: 10.1111/cgf.14794.

[10] C. V. R. Hütter, C. Sin, F. Müller, and J. Menche, "Network Cartographs for Interpretable Visualizations," *Nature Computational Science*, vol. 2, no. 2, pp. 84–89, Feb. 2022, doi: 10.1038/s43588-022-00199-z.

[11] D. J. Watts and S. H. Strogatz, "Collective Dynamics of 'Small-World' Networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998, doi: 10.1038/30918.

[12] J. Miller, V. Huroyan, and S. Kobourov, "Balancing Between the Local and Global Structures (LGS) in Graph Embedding," *Graph Drawing and Network Visualization*, vol. 14465. Springer Nature Switzerland, Cham, pp. 263–279, 2023. doi: 10.1007/978-3-031-49272-3_18.

[13] Y. Lu, J. Corander, and Z. Yang, "Doubly Stochastic Neighbor Embedding on Spheres," *Pattern Recognition Letters*, vol. 128, pp. 100–106, Dec. 2019, doi: 10.1016/j.patrec.2019.08.026.

[14] G. E. Hinton and S. Roweis, "Stochastic Neighbor Embedding," in *Advances in Neural Information Processing Systems*, S. Becker, S. Thrun, and K. Obermayer, Eds., MIT Press, 2002.

[15] L. van der Maaten and G. Hinton, "Visualizing Data Using T-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[16] D. E. Knuth, "Two thousand years of combinatorics," in *Combinatorics: Ancient and Modern*, Oxford University Press, 2013, pp. 3–37.

[17] Y. Hu and L. Shi, "Visualizing large graphs," *WIREs Computational Statistics*, vol. 7, no. 2, pp. 115–136, 2015, doi: https://doi.org/10.1002/wics.1343.

[18] P. Eades, "A Heuristic for Graph Drawing," 1984. [Online]. Available: https://api.semanticscholar.org/CorpusID:63936426

[19] T. M. J. Fruchterman and E. M. Reingold, "Graph drawing by force-directed placement," *Software: Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991, doi: https://doi.org/10.1002/spe.4380211102.

[20] A. Noack, "An Energy Model for Visual Graph Clustering," in *Graph Drawing*, G. Liotta, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 425–436.

[21] I. Bruß and A. Frick, "Fast Interactive 3-D Graph Visualization," in *International Symposium Graph Drawing and Network Visualization*, 1995. [Online]. Available: https://api.semanticscholar.org/CorpusID:16094190

[22] Y. Hu, "Efficient, High-Quality Force-Directed Graph Drawing," *The Mathematica journal*, vol. 10, pp. 37–71, 2006, [Online]. Available: https://api.semanticscholar.org/CorpusID:14599587

[23] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," *nat*, vol. 324, no. 6096, pp. 446–449, Dec. 1986, doi: 10.1038/324446a0.

[24] D. Tunkelang, "A Numerical Optimization Approach to General Graph Drawing," p. , 1999.

[25] A. J. Quigley, "Large scale relational information visualization, clustering, and abstraction," PhD Thesis, 2001.

[26] W. S. Torgerson, "Multidimensional Scaling: I. Theory and Method," *Psychometrika*, vol. 17, no. 4, pp. 401–419, 1952, doi: 10.1007/BF02288916.

[27] J. B. Kruskal, "Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis," *Psychometrika*, vol. 29, pp. 1–27, 1964, [Online]. Available: https://api.semanticscholar.org/CorpusID:48165675

[28] T. Kamada and S. Kawai, "An algorithm for drawing general undirected graphs," *Information Processing Letters*, vol. 31, no. 1, pp. 7–15, 1989, doi: https://doi.org/10.1016/0020-0190(89)90102-6.

[29] E. Gansner, Y. Koren, and S. North, "Graph Drawing by Stress Majorization," 2004, p. . doi: 10.1007/978-3-540-31843-9_25.

[30] K. M. Hall, "An r-Dimensional Quadratic Placement Algorithm," *Management Science*, vol. 17, no. 3, pp. 219–229, Nov. 1970, doi: 10.1287/mnsc.17.3.219.

[31] Y. Koren, L. Carmel, and D. Harel, "ACE: A Fast Multiscale Eigenvectors Computation for Drawing Huge Graphs," 2002, pp. 137–144. doi: 10.1109/INFVIS.2002.1173159.

[32] J. F. Kruiger, P. E. Rauber, R. M. Martins, A. Kerren, S. Kobourov, and A. C. Telea, "Graph Layouts by T-SNE," *Computer Graphics Forum*, vol. 36, no. 3, pp. 283–294, 2017, doi: 10.1111/cgf.13187.

[33] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction." [Online]. Available: https://arxiv.org/abs/1802.03426

[34] L. van der Maaten, "Accelerating t-SNE using Tree-Based Algorithms," *Journal of Machine Learning Research*, vol. 15, no. 93, pp. 3221–3245, 2014, [Online]. Available: http://jmlr.org/papers/v15/vandermaaten14a.html

[35] G. C. Linderman, M. Rachh, J. G. Hoskins, S. Steinerberger, and Y. Kluger, "Fast Interpolation-based t-SNE for Improved Visualization of Single-Cell RNA-Seq Data," *Nature methods*, vol. 16, pp. 243–245, 2017, [Online]. Available: https://api.semanticscholar.org/CorpusID:34472149

[36] T. M. J. Fruchterman and E. M. Reingold, "Graph Drawing by Force-directed Placement," *Software: Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, Nov. 1991, doi: 10.1002/spe.4380211102.

[37] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function Using NetworkX," presented at the Python in Science Conference, Pasadena, California, June 2008, pp. 11–15. doi: 10.25080/TCWV9851.

[38] E. Gansner and S. North, "An Open Graph Visualization System and Its Applications to Software Engineering," *Software: Practice and Experience*, vol. 30, p. , 2003, doi: 10.1002/1097-024X(200009)30:113.3.CO;2-E.

[39] M. Jacomy, T. Venturini, S. Heymann, and M. Bastian, "ForceAtlas2, a Continuous Graph Layout Algorithm for Handy Network Visualization Designed for the Gephi Software," *PLoS ONE*, vol. 9, 2014, [Online]. Available: https://api.semanticscholar.org/CorpusID:14763635

[40] M. Bastian, S. Heymann, and M. Jacomy, "Gephi: An Open Source Software for Exploring and Manipulating Networks," *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, no. 1, pp. 361–362, Mar. 2009, doi: 10.1609/icwsm.v3i1.13937.

[41] R. Ahmed, C. Erten, S. Kobourov, J. Lotz, J. Miller, and H. Taraz, "Size Should not Matter: Scale-invariant Stress Metrics." [Online]. Available: https://arxiv.org/abs/2408.04688

[42] K. Smelser, J. Miller, and S. Kobourov, ""Normalized Stress" is Not Normalized: How to Interpret Stress Correctly." [Online]. Available: https://arxiv.org/abs/2408.07724